Câu 1: Sự khác nhau giữa Value Types và Reference Types (Stack vs Heap)

Trong C#, các kiểu dữ liệu được chia thành hai nhóm chính với cách quản lý vùng nhớ hoàn toàn khác nhau:

- **Value Types (Kiểu giá trị):**
  - **Bản chất lưu trữ:** Biến chứa trực tiếp dữ liệu thực sự.
  - **Vị trí vùng nhớ:** Thường được lưu trên Stack (khi là biến cục bộ trong phương thức). Tuy nhiên, nếu là thuộc tính hoặc trường của một Class, dữ liệu của nó sẽ nằm trên Heap cùng với đối tượng đó.
  - **Cơ chế sao chép:** Khi gán biến này cho biến khác, dữ liệu được sao chép toàn bộ (Copy-by-value). Mọi thay đổi ở biến mới không làm ảnh hưởng đến biến cũ.
  - **Quản lý bộ nhớ:** Được quản lý tự động theo cơ chế LIFO của Stack và giải phóng ngay khi thoát khỏi scope.
  - **Các kiểu ví dụ:** int, float, bool, char, DateTime, struct, enum.
- **Reference Types (Kiểu tham chiếu):**
  - **Bản chất lưu trữ:** Biến chỉ chứa địa chỉ con trỏ (tham chiếu) trỏ tới nơi lưu dữ liệu thực sự trên Heap.
  - **Vị trí vùng nhớ:** Dữ liệu thực sự (Object Data) luôn nằm trên Heap, còn con trỏ địa chỉ chứa trong biến cục bộ sẽ nằm trên Stack.
  - **Cơ chế sao chép:** Khi gán biến này cho biến khác, chỉ có địa chỉ tham chiếu được sao chép (Copy-by-reference). Cả hai biến lúc này cùng trỏ vào một vùng nhớ trên Heap.
  - **Quản lý bộ nhớ:** Do Garbage Collector (GC) tự động thu gom và dọn dẹp trên Heap khi không còn biến nào trỏ tới.
  - **Các kiểu ví dụ:** string, object, class, interface, delegate, các mảng (int[],...).

Câu 2: Tính năng Init-only Properties (init) trong C# 9/10

init là từ khóa thay thế cho **set**, cho phép thuộc tính chỉ được phép gán giá trị tại thời điểm khởi tạo đối tượng, sau đó đối tượng sẽ trở thành **Immutable (bất biến)**.

1. Điểm khác biệt so với 'set' thông thường:

- **Thuộc tính có set:** Giá trị của thuộc tính có thể bị thay đổi tự do ở bất kỳ đâu và bất kỳ lúc nào trong suốt vòng đời của đối tượng.
- **Thuộc tính có init:** Giá trị chỉ được phép gán trong Constructor hoặc thông qua cú pháp Object Initializer. Khi quá trình khởi tạo hoàn tất, việc cố tình gán lại giá trị sẽ gây ra lỗi biên dịch (Compile-time error).

2. Trường hợp sử dụng thực tế:

Thích hợp cho các Data Transfer Objects (DTO), các lớp dữ liệu cấu hình (Configuration Models) hoặc các Entity chỉ đọc. Tính năng này giúp đảm bảo tính toàn vẹn của dữ liệu, tránh việc dữ liệu bị thay đổi vô tình ở các tầng xử lý khác

Câu 3: Phân biệt phương thức virtual (Lớp cha) và override (Lớp con)

Trong tính Đa hình (Polymorphism), **virtual** và **override** là cặp từ khóa đi kèm với nhau để cho phép lớp con định nghĩa lại hành vi từ lớp cha:

- **virtual (Khai báo tại Lớp cha):**
  - Khai báo rằng phương thức này có triển khai mặc định, nhưng cho phép các lớp con có quyền ghi đè (override) lại nếu cần.
  - Bắt buộc phải có phần thân phương thức triển khai mặc định.
  - Lớp con không bắt buộc phải ghi đè; nếu không ghi đè, nó sẽ dùng logic mặc định của lớp cha.
- **override (Khai báo tại Lớp con):**
  - Khai báo ở lớp con để thực sự thay thế (ghi đè) triển khai của phương thức virtual từ lớp cha.
  - Cung cấp logic mới phù hợp với đối tượng lớp con.
  - Khi gọi phương thức qua một con trỏ lớp cha trỏ tới đối tượng lớp con, C# sử dụng cơ chế Dynamic Binding (Ràng buộc động) để gọi phương thức override của lớp con thay vì virtual của lớp cha.

Câu 4: Tại sao thành phần static không thể truy xuất qua một Object Instance?

Thành phần khai báo với từ khóa **static** thuộc về bản thân Lớp (Type/Class) chứ không thuộc về bất kỳ thể hiện (Instance) cụ thể nào được tạo ra từ toán tử **new**. Trình biên dịch C# cấm truy cập qua con trỏ đối tượng vì các lý do sau:

1. **Cơ chế quản lý bộ nhớ độc lập:** Thành phần static được cấp phát bộ nhớ duy nhất một lần trong High Frequency Heap ngay khi Class được nạp vào CLR. Khi bạn tạo đối tượng mới bằng new, vùng nhớ Heap của đối tượng đó chỉ chứa các dữ liệu non-static riêng biệt.
2. **Không sử dụng tham chiếu 'this':** Các phương thức hay thuộc tính static không nhận con trỏ 'this' ngầm định. Chúng không hoạt động trên dữ liệu riêng của từng đối tượng nên không có lý do để gắn liền với một thể hiện cụ thể.
3. **Triết lý thiết kế ngôn ngữ (Code Clarity):** Trình biên dịch C# chủ động ngăn chặn cú pháp instance.StaticMember nhằm mục đích rõ ràng về mặt ngữ nghĩa, tránh gây hiểu nhầm cho lập trình viên rằng thao tác đó đang làm thay đổi trạng thái riêng của đối tượng đó.